Real-Time Dense Map Matching with Naive Hidden Markov Models

G Harish reddy (PES1UG24AM366) | SHARANABASAVA H (PES1UG24AM399)

UE24CS352A Machine Learning | Section: Not provided | Problem: Not provided

Problem statement

Assign noisy vehicle GPS observations to directed road arcs in real time, without shortest-path searches. The objective is to implement the naive HMM approach of Ahres et al. and measure how delaying irrevocable predictions by N observations changes accuracy for a window of T observations.

Dataset

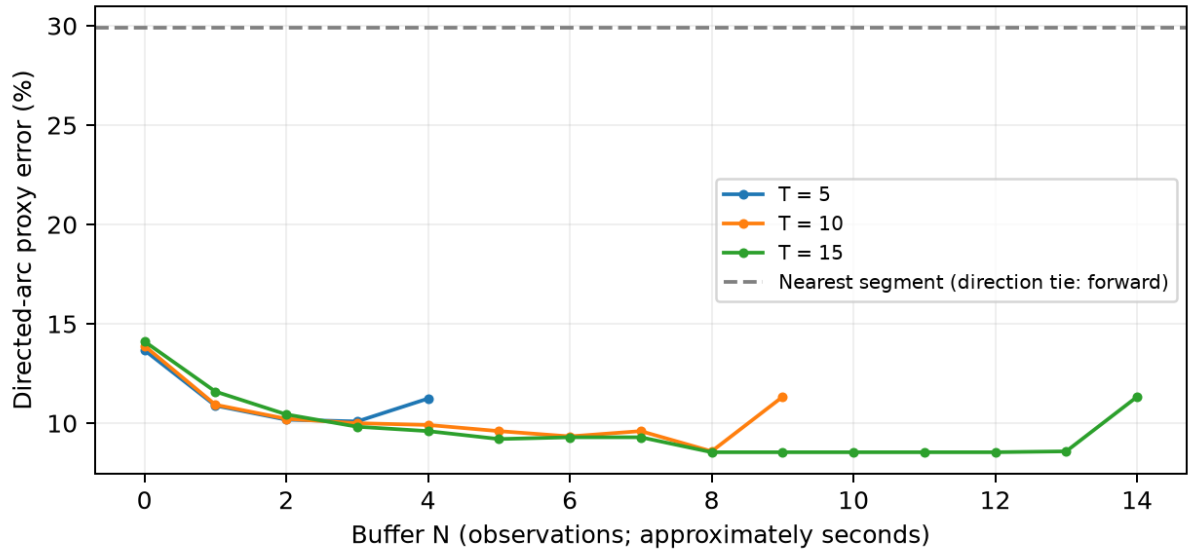
The public Newson-Krumm Seattle benchmark contains 7,531 GPS readings, 158,167 road edges (306,182 directed arcs), and a 576-edge reference route. The nominal 1 Hz trace has four sampling gaps and spans 7,611 seconds. Longitude/latitude are projected to UTM zone 10N. Speed and heading use a causal five-reading displacement. The first 4,518 points train weights, the next 753 select T/N, and the final 2,260 form the test set (60/10/30 split).

Reference-label construction

The benchmark supplies an ordered route, not independent per-point labels. Offline dynamic programming minimizes distance to route arcs under monotone progress, allowing stay/advance by up to eight edges and adding a one-metre penalty per advanced edge. Repeated route occurrences remain distinct. These derived labels are evaluation proxies; the median GPS-to-aligned-arc distance is 2.75 m. Candidate coverage on test is 99.91%.

Approach

Candidate states are all directed arcs within 50 m of a GPS point, found using a spatial index. Emissions combine Gaussian point-to-road density (sigma=10 m), directed heading agreement, and a high-speed/speed-limit constraint. Transitions combine a committed-history backtrack penalty and reciprocal GPS-versus-arc-midpoint distance discrepancy. Finite-window Viterbi maximizes summed log potentials, clamps already committed states at their actual indices, and commits state t-N when reading t arrives. At EOF, the remaining N states are flushed. No road-network shortest path or connectivity check is used.



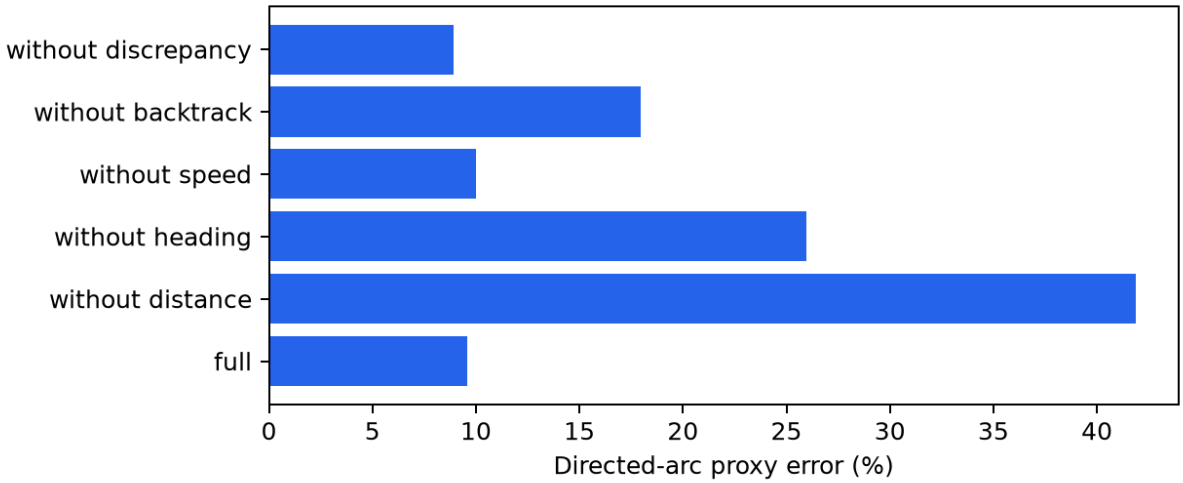
Model	T / N	Directed error	Edge error
Zero delay	10 / 0	13.89%	8.67%
Validation-selected	10 / 7	9.60%	4.60%
Nearest segment	- / 0	29.91%	8.01%

Learning and implementation

Python modules separate loading, candidates, features, decoding, learning, evaluation, and visualization. Balanced L1 LinearSVC models learn emission then transition coefficients independently from reference labels. Ten expanding time folds, with a 15-observation gap, select C from 0.1, 1, 10 using F1 (selected emission C=10, transition C=10). Standardization is fitted within each fold. Positive raw-space coefficients are L1-normalized; negatives are clipped, the intercept is discarded, and scores have an epsilon floor. These are uncalibrated positive potentials. Training uses reference history for backtrack features; inference uses committed history.

Results

Validation selects T=10, N=7 (validation directed error 5.05%). On 2,260 held-out points, directed-arc error is 9.60%, edge error 4.60%, and online-only directed error 9.63%; seven EOF points are flushed. Nearest-road edge error is 8.01%. Directed baseline error is 29.91%, but its forward-direction tie rule disadvantages that baseline. Matching takes 0.537 s/1,000 points on this machine, excluding loading and visualization.



Conclusions

Buffering reduces error relative to zero delay, but the test minima differ by window: N=3, 8, and 8 for T=5, 10, and 15. The curves are not uniformly convex, and the minimum is not always T/2. Removing distance discrepancy improves directed error from 9.60% to 8.94%; this feature is not universally helpful. Other ablations worsen error. Results differ from the paper because the benchmark, labels, learned weights, and decoder choices differ. Disconnected predictions and proxy-label bias remain limitations. Future work should use independent point labels, additional routes, causal noise filtering, and better turn/connectivity features.

Reproducibility

Install requirements; download the benchmark; run python -m src.train_weights, python -m src.evaluate, and python run_demo.py --T 10 --N 7. Saved CSV metrics feed this PDF and the deck. Source repository URL has not been supplied; the GitHub deliverable still requires publishing and collaborator access.

References

Ahres, Jambulapati, Janssen & Kangaspunta, Real-Time Dense Map Matching with Naive Hidden Markov Models: Delay versus Accuracy, supplied CS229 paper. Newson & Krumm (2009), Hidden Markov Map Matching Through Noise and Sparseness, ACM GIS, pp. 336-343, doi:10.1145/1653771.1653818. Forney (1973), The Viterbi Algorithm, Proceedings of the IEEE 61(3), pp. 268-278.